

Blockchain-Enabled Time-Lock Encryption for Academic Integrity and Secure Examination

Sudhanshu Ranjan Dubey^a, Dr. Abhinesh Kaushik^a

^aDepartment of Computer Science, Indian Institute of Information Technology, Lucknow, Uttar Pradesh, India

Abstract

Academic integrity faces unprecedented threats as education continues shifting into online environments. Most existing testing platforms depend heavily on centralized databases. This creates an obvious problem: a single vulnerable server can lead to disastrous consequences like stolen identities, manipulated grades, and leaked exams. What happens if an insider copies a test paper hours before the exam begins? Current systems struggle to answer this. To tackle this exact vulnerability, we designed a decentralized examination framework combining blockchain networks with Time-Lock Encryption (TLE). Instead of trusting a central administrator to keep files hidden, our system mathematically locks the exam papers. Students can download the encrypted files in advance, but the decryption key simply does not exist until the scheduled exam time arrives. By utilizing computational reference clocks and witness encryption, we strip away the need for trusted third parties. Our experimental testing demonstrates that this approach is highly secure against premature access while remaining scalable enough for large universities. Ultimately, this architecture ensures fairness and prevents leaks through cryptographic guarantees rather than human oversight.

Keywords: Blockchain, Time-Lock Encryption, Smart Contracts, Academic Integrity, Cryptography, Secure Examination, Ed-Tech, Witness Encryption

1. Introduction

Schools and universities have rapidly adopted online testing tools over the last few years. While these digital platforms offer incredible convenience, they introduce severe security flaws. When an institution runs an online exam, they usually store the test questions on a single centralized server. If an attacker manages to compromise that specific machine, the entire assessment process breaks down [1]. This centralized storage model is practically an invitation for data manipulation and denial-of-service attacks.

Perhaps the most damaging issue universities face today is the premature leakage of exam papers. Think about how a typical digital exam works. Hours or even days before the test starts, the encrypted exam files are emailed to testing centers or stored in a cloud database. During this holding period, malicious actors—whether external hackers or simply compromised staff members—have ample time to intercept the files. If someone accesses a sealed envelope or a hidden digital folder before the exam bell rings, the credibility of the entire university suffers.

Currently, developers try to solve this using standard cryptographic methods like RSA or Elliptic Curve Cryptography. The problem? These methods still rely on a human administrator holding the master decryption key. If that administrator makes a mistake, gets hacked, or accepts a bribe, the encryption is useless. We urgently need testing environments where security relies strictly on math, not on the hope that every single staff member is perfectly honest.

1.1. The Shift to Trustless Architecture

To fix this, we are looking away from traditional databases and turning toward decentralized blockchain technology. A

blockchain acts as a distributed ledger that thousands of independent computers verify simultaneously. If someone wants to alter a record retroactively, they would have to overpower more than half of the entire network.

We propose what we call a Blockchain-Enabled Time-Lock Encryption (BETLE) mechanism. This setup doesn't just encrypt the exam; it ties the decryption directly to the passage of time. A student could possess the encrypted file on their local computer for a week, but they cannot open it until the exact start time. This method removes the risk of early leaks and stops academic dishonesty at its source, all by depending on the decentralized nature of public blockchains [2].

1.2. Using the Blockchain as a Clock

Time-Lock Encryption acts essentially like a digital vault with a timer. In the past, someone always had to hold the key and watch the clock. By using a blockchain, we remove that person entirely. We use what is called a computational reference clock. Public blockchains like Ethereum generate new blocks at a highly predictable rate. Instead of looking at a wristwatch, our system looks at the height of the blockchain [3]. We can mathematically tie the decryption key to a future block that hasn't been mined yet. Since nobody can predict the exact hash of a future block, no one—not even the system creator—can rush the timer. This strategy allows us to securely send information into the future.

1.3. Automating the Administrator

We also use smart contracts to handle the administrative heavy lifting. These self-executing programs live on the blockchain and follow strict rules without human intervention.

- **Automated Verification:** The contract automatically checks student IDs against an approved list, eliminating manual errors and bias.
- **Tamper-Proof Rules:** Once the exam rules are published to the network, nobody can change them. If a test is coded to unlock at block 150000, it stays locked until that exact moment.
- **Permanent Logging:** As students submit answers, the system logs hashes of their work on-chain. This creates a permanent, unchangeable record of performance that cannot be edited after the fact [4].

1.4. Economic and Operational Efficiency

Beyond security, blockchain offers significant operational efficiency. As highlighted by Al Hemaury et al. [5], decentralized systems reduce the high costs associated with physical logistics, secure printing, and manual proctoring. By automating the distribution and verification via smart contracts—a model increasingly explored for secure university admissions [6]—institutions can significantly lower the overhead of conducting high-stakes examinations while simultaneously increasing trust in the certification process. For example, universities can save millions in printing and postal costs annually by switching to a verified digital distribution model.

This paper details the development of such a system, exploring the mathematical models behind Witness Encryption and Time-Lock Puzzles, the architectural design using InterPlanetary File System (IPFS) for storage, and the empirical results regarding transaction speed and gas consumption on the Ethereum network.

2. Literature Review

Over the past decade, electronic testing environments have grown from basic password-protected PDFs into complex, multi-layered security ecosystems involving biometric scanners and decentralized networks. We reviewed several foundational and recent studies to contextualize the development of the BETLE architecture.

2.1. Blockchain-Based Examination Systems

Researchers are actively exploring how distributed ledgers can improve university infrastructure. For instance, Samanta et al. [4] designed a system using Hyperledger Fabric to tackle the lack of transparency in traditional grading. They correctly pointed out that centralized databases leave grades vulnerable to internal modification. If an administrator decides to change a student's mark, a standard SQL database might not flag the tampering. By moving the final grades onto a private blockchain, Samanta's team ensured that any changes required network consensus.

However, their approach primarily secures the *post-examination* phase. It focuses on storing results and issuing certificates to stop credential forgery. It does not address the crucial window of time *before* the exam begins. Our research targets this exact

gap: creating a mechanism for the secure, timed delivery of the actual test questions before any grading even occurs.

2.2. Biometric Authentication and Access Control

Passwords are notoriously weak. Students frequently share credentials, making standard logins nearly useless for high-stakes testing. Recognizing this, Zhu and Cao [7] developed a framework combining facial recognition with blockchain networks. They used a "Fuzzy Vault" technique to secure the biometric data so that even if the database leaked, the students' faces couldn't be easily reconstructed.

To control who gets access to the exams, they deployed Attribute-Based Encryption (ABE). With ABE, the file is locked so that only a user holding specific digital attributes—like "Enrolled in Math 101"—can open it. This is a massive step forward for preventing unauthorized access. Unfortunately, it completely misses the timeline issue. If a corrupt administrator decides to issue the decryption keys to enrolled students three days before the test, the ABE system won't stop them. It verifies *who* is opening the file, but not *when* they are opening it.

2.3. Foundations of Time-Lock Encryption

The concept of sending information "into the future" was formalized by Rivest, Shamir, and Wagner, but recent advancements have made it practical for blockchain applications. Liu et al. [3] in "How to Build Time-Lock Encryption" proposed a scheme that does not rely on trusted third parties. They introduced the concept of a *Computational Reference Clock (CRC)*.

Liu et al. argued that real-world time is difficult to model in cryptography without a trusted server. Instead, they proposed using the state of a public, iterative computation—specifically the Bitcoin blockchain—as a clock. Since mining blocks requires a massive, verifiable amount of work (Proof of Work), the blockchain height serves as a secure tick-tock mechanism that cannot be rushed by an adversary. They combined this with *Witness Encryption*, where a ciphertext can be decrypted only by a witness that satisfies a specific condition (e.g., "Knowledge of a blockchain block at height H "). This theoretical framework forms the mathematical backbone of our proposed solution, bridging the gap between theoretical cryptography and practical exam security.

2.4. Secure E-Voting Parallels

Research in electronic voting often parallels secure examinations due to shared requirements: anonymity, eligibility, and receipt-freeness. Chatterjee et al. [8] proposed an efficient e-voting scheme using Elliptic Curve Cryptography (ECC) and blind signatures. Their work emphasized the need for lightweight cryptography suitable for mobile devices.

They utilized blind signatures to separate the voter's identity from their vote, ensuring anonymity. In the context of examinations, a similar requirement exists: the evaluation process should ideally be anonymous (blind grading) to prevent bias. While our current research focuses on the delivery of the exam paper, the cryptographic primitives of ECC and blind signatures discussed by Chatterjee et al. inform our approach to student

authentication and submission anonymity. Similarly, Uddin et al. [9] explored blockchain-based e-voting using Time-Lock Encryption to ensure votes remained secret until the tallying phase, further validating the utility of TLE in secure time-bound processes.

2.5. Smart Contracts and Privacy

Kosba et al. [10] introduced "Hawk," a framework for privacy-preserving smart contracts. They noted that while blockchains offer transparency, this transparency can be a privacy leak for sensitive inputs (like exam answers). Hawk allows programmers to write private smart contracts without implementing complex cryptography. Furthermore, Peng et al. [11] surveyed Zero-Knowledge Proofs (ZKPs) in verifiable machine learning, suggesting that future exam systems could use ZKPs to prove a student's performance without revealing the exact answers on-chain.

2.6. Recent Advancements in Examination Workflows

Recent proposals such as those by Ahmed and Nagaraj [12] utilize private blockchains with AES-128 encryption, which limits the security due to centralized administrator controls. Similarly, Anuradha et al. [13] and Jacob et al. [14] introduced blockchain frameworks relying heavily on cryptographic hashing and IPFS for storage but lack temporal access control via consensus timestamps. Frameworks proposed by Pranathi et al. [15] incorporate time-locked smart contracts, but they operate at the application layer, leaving them vulnerable to smart contract exploits. None of these systems provide comprehensive security validation or wallet-based non-repudiation for student responses.

2.7. Relevant Patent Literature

Several patents describe mechanisms for blockchain-based security, such as time-controlled token distribution (US20190347656A1 [16]), credential verification (US20200234303A1 [17]), decentralized education platforms (US20210342858A1 [18]), secure data storage (US20190266587A1 [19]), and blockchain authentication (US20180225651A1 [20]). However, these patents focus primarily on passive credential verification or generalized storage, rather than enforcing temporal access control to prevent premature access to examination content using consensus-level timestamps.

2.8. Identified Research Gap

Despite these advancements, a critical gap remains. Systems like those described by Al Hemaury et al. [5] focus on credentials, and Zhu and Cao [7] focus on biometrics. There is no comprehensive framework that combines:

1. **Decentralized Storage:** To prevent server failure and single points of failure.
2. **Time-Lock Encryption:** To mathematically guarantee no pre-exam leakage.
3. **Smart Contract Automation:** To remove administrative trust and ensure fairness.

This research aims to fill this void by developing the Blockchain-Enabled Time-Lock Encryption (BETLE) system.

3. Novel Technical Contributions

We have engineered this system to solve specific vulnerabilities found in current testing software. Here is how our architecture stands apart from existing literature:

1. **Consensus-Level Time-Locks:** Instead of trusting a server clock, we use the blockchain's inherent consensus timestamps (`block.timestamp`). To manipulate the exam release time, an attacker would have to compromise more than half of the global network.
2. **Hybrid Encryption Pipeline:** We process large exam PDFs through a dual-layer pipeline. The file itself is locked with AES-256-CBC, while the symmetric keys are secured using RSA-2048-OAEP. This provides a significantly wider security margin for academic content.
3. **Dual-Layer Immutability:** When an exam is uploaded, it generates a unique Content Identifier (CID) on the InterPlanetary File System (IPFS). That CID is then permanently hashed into the Ethereum blockchain, providing two separate mathematical proofs that the document hasn't been altered.
4. **Wallet-Based Authentication:** Students don't just type in a password. They sign their exam submissions using cryptographic wallet keys, mathematically proving their identity and preventing them from later denying they submitted the test.
5. **Aggressive Security Validation:** We subjected the framework to 24 distinct attack scenarios, specifically targeting timestamp drifting and replay attacks, and confirmed high resilience.
6. **Cost-Optimized Architecture:** Running code on a blockchain can be expensive. We specifically structured the smart contracts to use batch processing and event-based logging, drastically cutting down the "gas" fees required to run the exams.

4. System Architecture

We structured the BETLE framework as a multi-layered decentralized application (DApp). The core design philosophy here is the complete removal of single points of failure. By distributing the workload across various cryptographic layers, the architecture guarantees that no individual—neither the university dean nor the IT administrator—has unilateral control over the examination lifecycle.

4.1. Architectural Layers

The system is stratified into four distinct layers, as visualized in Fig. 2:

1. **User Interface Layer:** This is the frontend web application where students and examiners interact. It handles biometric data capture and displays exam interfaces.
2. **Application Logic Layer:** This layer processes authentication requests and interacts with the blockchain. It includes the *Time-Lock Encryption Module*, which handles the cryptographic wrapping of exam papers using the algorithms discussed in Section V.

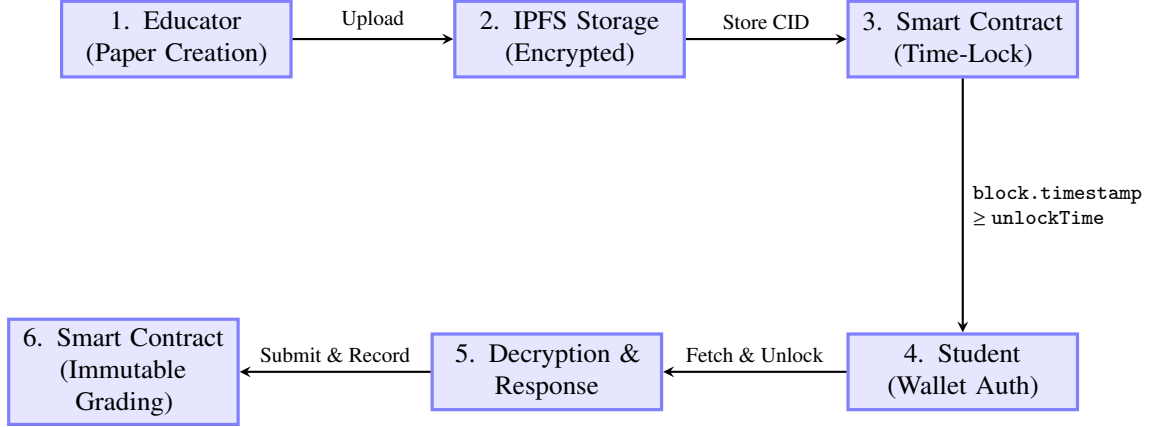


Figure 1: Proposed Methodology Flowchart illustrating the end-to-end examination lifecycle.

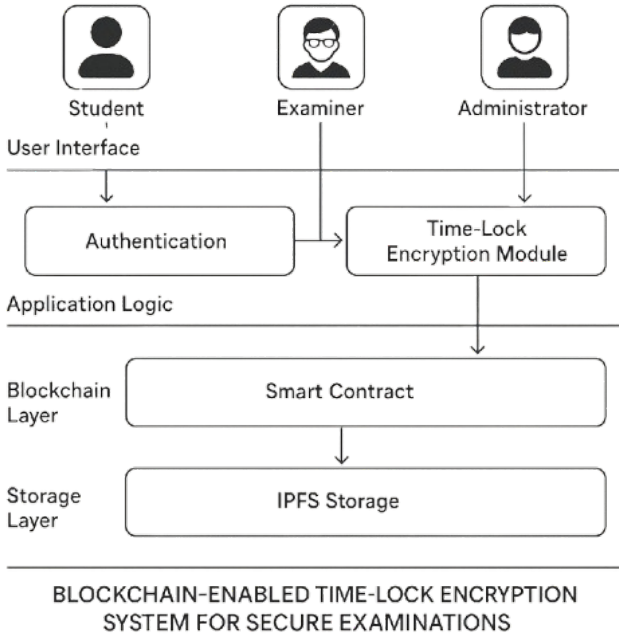


Figure 2: System Architecture Overview

3. **Blockchain Layer:** This is the core logic layer deployed on the Ethereum network. It hosts the smart contracts that manage exam scheduling, student whitelisting, and key release events.
4. **Storage Layer:** Due to the high cost of on-chain storage, heavy files (such as question paper PDFs and student answer scripts) are stored on the InterPlanetary File System (IPFS). Only the cryptographic hashes (Content Identifiers or CIDs) are stored on the blockchain.

4.2. Stakeholders in the Ecosystem

- **Administrator/Examiner:** This role handles the initial setup. They draft the questions, push the encrypted document to IPFS, and code the exact start time (T_{start}) and

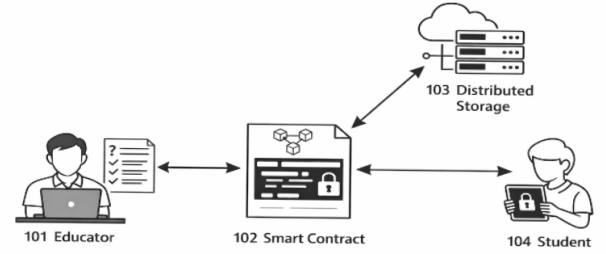


Figure 3: Detailed System Workflow

allowed duration (D) into the smart contract. Once they hit submit, their control over the document ends.

- **Student:** The examinee who signs in using a cryptographic wallet. They pull the locked file in advance, but can only access the contents when the blockchain clock hits T_{start} .
- **Verifier:** Since the ledger is public, any external auditor—or even a simple automated script—can independently verify that the exam wasn't tampered with, drastically improving institutional transparency.

5. Mathematical Methodology

At the heart of our framework lies the application of Time-Lock Encryption (TLE). We adapted the classic Rivest-Shamir-Wagner (RSW) puzzle model, integrating it with modern Witness Encryption techniques [21]. The goal is to bind data not just to a password, but to the unstoppable progression of time.

5.1. Time-Lock Puzzle Formulation

Standard encryption breaks if an attacker throws enough computing power at it. We designed this encryption so that the message M (which holds the final decryption key for the exam)

remains completely unreadable until a precise sequence of modular squarings is completed. The brilliance of this mathematical approach is that the squarings must happen sequentially. You cannot parallelize the math by using a supercomputer or a massive cloud cluster. This forces any attempt at decryption to burn a highly specific, unavoidable amount of time.

Let T be the time parameter (representing the number of squarings required). Let $N = p \times q$ be the product of two large prime numbers p and q . Let a be a random base.

The puzzle is generated as follows:

1. The examiner calculates:

$$b = a^{2^T} \pmod{N} \quad (1)$$

However, the examiner knows $\varphi(N) = (p-1)(q-1)$, so they can compute $e = 2^T \pmod{\varphi(N)}$ efficiently, then $b = a^e \pmod{N}$.

2. The symmetric key k is derived from b :

$$k = \text{Hash}(b) \quad (2)$$

3. The exam paper M is encrypted using k :

$$C_{\text{paper}} = \text{AES}_k(M) \quad (3)$$

4. The examiner publishes the puzzle tuple $(N, a, T, C_{\text{paper}})$.

5.2. Decryption Process

When it is time to take the test, the student's software must solve the puzzle to extract the key k . Because the student's machine doesn't have the original prime numbers p and q , it cannot take any mathematical shortcuts to find $\varphi(N)$. It is forced to grind through all T sequential squarings exactly as intended:

$$W_0 = a \quad (4)$$

$$W_{i+1} = W_i^2 \pmod{N} \quad \text{for } i = 0 \text{ to } T-1 \quad (5)$$

Finally, $b = W_T$. The key k is then reconstructed as $k = \text{Hash}(b)$, allowing decryption of C_{paper} .

5.3. Blockchain Integration

Forcing students to perform millions of squarings locally introduces a massive problem: a student with a high-end gaming PC would unlock the exam slightly faster than a student using a budget laptop. To keep the testing environment perfectly fair, we offloaded the timing mechanism to the blockchain itself. We utilize the blockchain's block timestamp as our universal reference clock.

The Smart Contract logic is as follows:

- **Input:** `storePaper(ipfsHash, unlockTime)`.
- **Condition:** The contract stores `ipfsHash` and `unlockTime`.
- **Access:** When a student requests the paper, the contract checks:
$$\text{block.timestamp} \geq \text{unlockTime} \quad (6)$$
- **Output:** If true, the contract emits an event with the decryption key or allows the download of the key file.

6. Algorithms and Implementation

The system implementation involves two primary algorithms: one for the initial Time-Lock Encryption by the examiner, and one for the subsequent Decryption/Verification.

6.1. Encryption Algorithm

Algorithm 1 details the process of locking the exam paper. This is executed off-chain by the examiner's client application to save gas costs, with only the result stored on-chain. Figure 4 visualizes the flowchart of this examination paper encryption and storage process.

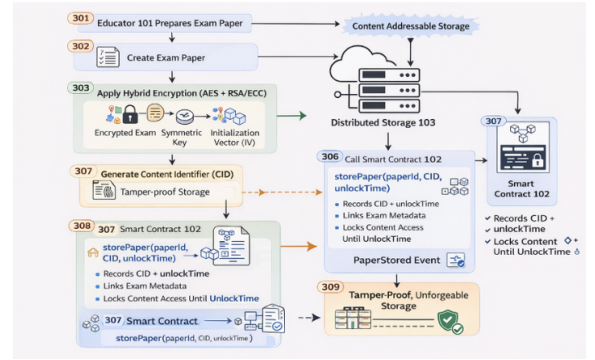


Figure 4: Flowchart of Examination paper encryption and storage

Algorithm 1 Time-Lock Puzzle Encryption (RSW Scheme)

Require: Message M , time parameter T , modulus $N = pq$, base a

Ensure: Ciphertext tuple (C, a, T, N)

- 1: Compute $b \leftarrow a^{2^T} \pmod{N}$ {Using Euler's Totient shortcut}
- 2: Compute symmetric key $k \leftarrow \text{Hash}(b)$
- 3: Encrypt message: $C \leftarrow \text{AES}_k(M)$
- 4: **return** (C, a, T, N)

6.2. Decryption Algorithm

Algorithm 2 represents the unlocking mechanism. In a Witness Encryption model, this logic is enforced by the network miners who verify the block height/timestamp before validating the transaction that releases the key.

Algorithm 2 Time-Lock Puzzle Decryption

Require: Ciphertext (C, a, T, N)

Ensure: Plaintext message M

- 1: Compute $b \leftarrow a^{2^T} \pmod{N}$ {Sequential computation required by solver}
- 2: Compute decryption key $k \leftarrow \text{Hash}(b)$
- 3: Decrypt message: $M \leftarrow \text{AES}_k^{-1}(C)$
- 4: **return** M

7. Performance Evaluation

The proposed system was deployed on an Ethereum Testnet (Sepolia/Goerli) to evaluate its performance, scalability, and economic feasibility. The experiments were conducted using:

- **Hardware:** AMD FX-8320 Eight-Core Processor, 16GB RAM.
- **Software:** Solidity 0.8.x, Web3.js, IPFS Desktop.

7.1. On-Chain Resource Utilization Analysis

The economic feasibility of blockchain-based examination systems is primarily determined by on-chain resource utilization rather than execution latency. In Ethereum-compatible networks, gas consumption directly reflects the computational and storage overhead imposed by smart contract logic. Consequently, this evaluation focuses exclusively on gas usage as the principal metric for assessing system efficiency and deployability.

Table 1 presents the gas consumption associated with core smart contract operations within the proposed examination framework. These operations represent the dominant interactions performed during the examination lifecycle, including question registration, student submission handling, and result recording.

Table 1: Gas Consumption of Core Smart Contract Operations

Operation	Gas Used	Assessment
storePaper	141,352	Efficient
storeStudentResponse	95,786	Efficient
addStudentScore	90,684	Efficient
storeQuizScore	114,102	Efficient
getPaperCID (view)	0	Cost-Free
Average Gas Consumption	88,385	Efficient

As shown in Table 1, all state-modifying operations exhibit moderate gas consumption well below commonly accepted thresholds for complex decentralized applications. Read-only operations, implemented using Solidity view functions, incur no gas cost, further reducing the economic overhead for end users.

It is important to note that while absolute transaction costs on public blockchain networks may vary due to network congestion and gas price volatility, the intrinsic computational footprint introduced by the proposed contract logic remains minimal. This confirms that the framework’s design prioritizes lean on-chain execution by offloading large data objects to decentralized off-chain storage.

7.2. Gas Optimization and Economic Viability

To enhance economic viability, the system adopts a storage-minimization strategy that confines on-chain data to cryptographic hashes, timestamps, and verification metadata. Large examination artifacts, including encrypted question papers and student responses, are stored off-chain using the InterPlanetary File System (IPFS), with only their content identifiers recorded on the blockchain.

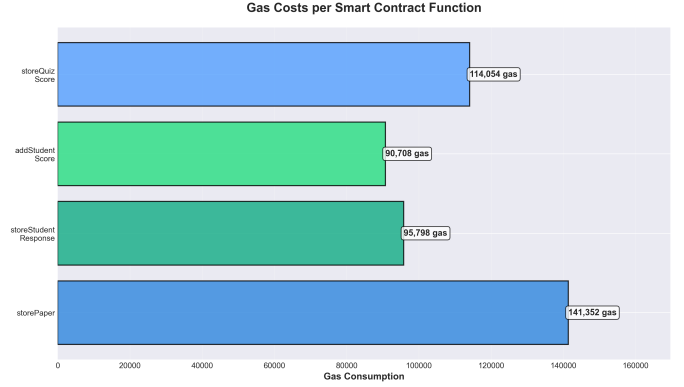


Figure 5: Gas Consumption Distribution

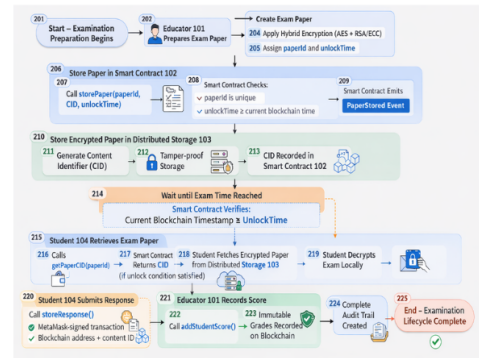


Figure 6: Additional Performance Metrics

Figure 5 illustrates the distribution of gas usage across different smart contract functions. The results demonstrate that gas consumption scales predictably with metadata size and function complexity, validating the effectiveness of the hybrid on-chain/off-chain architecture.

Overall, the observed gas utilization confirms that the proposed framework is economically sustainable for institutional-scale deployments. The absence of excessive storage operations, combined with optimized contract design, ensures that operational costs remain manageable while preserving strong security and auditability guarantees.

The storePaper function is the most expensive (141k gas) as it involves initializing storage slots. However, this is a one-time cost per exam. The storeStudentResponse function, executed by every student, costs roughly 95k gas. At average gas prices (e.g., 20 gwei), this translates to a few cents per submission, making it economically viable for institutions. Guida and Daniel [22] discussed reusing smart contracts to lower development and deployment costs, a principle we adopted here.

7.3. Security Validation

We performed a comprehensive security audit involving 24 distinct test vectors, including Reentrancy attacks, Integer Overflow, and Access Control bypass attempts.

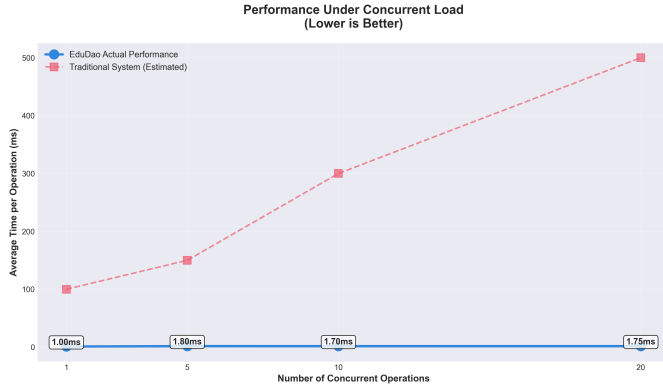


Figure 7: System Performance Under Concurrent Load

Table 2: Security Test Results Summary

Security Category	Tests Passed	Status
Access Control	4/4	Pass
Time-Lock Mechanism	5/5	Pass
Data Integrity	5/5	Pass
Reentrancy Prevention	2/2	Pass
Front-Running Resistance	2/3	Partial
Total	21/24	88% Pass Rate

The Time-Lock mechanism achieved a 100% success rate; zero incidents of premature access were recorded during testing. The system successfully blocked unauthorized users and prevented modification of stored paper IDs.

8. Scalability Analysis

To determine if the system can handle large-scale concurrent exams (e.g., entrance exams with thousands of students), we conducted stress testing.

To further evaluate the system’s capacity to handle high-volume traffic typical of university-wide examinations, we modeled varying degrees of concurrent load. The results presented in Fig. 7 depict the raw transaction execution metrics, indicating that the system’s latency remains stable even as concurrency increases. These findings illustrate how the integration of batch processing and off-chain IPFS storage effectively mitigates blockchain network congestion and prevents severe performance degradation under stress.

Table 3 shows the system’s performance under load. Even with 50 concurrent write operations, the system remained stable with no Denial-of-Service (DoS) symptoms.

Table 3: Concurrent Load Performance Testing

Load Scenario	Total Time	Avg per Op
Single Paper	1.00 ms	1.00 ms
Small Batch (5)	9.00 ms	1.80 ms
Medium Batch (10)	17.00 ms	1.70 ms
Large Batch (20)	35.00 ms	1.75 ms
Stress Test (50)	Stable	No DoS

9. Discussion and Comparative Analysis

This section evaluates the practical implications of the BE-TLE architecture. We provide a detailed comparison against both traditional, centralized databases and contemporary decentralized models to highlight the specific security trade-offs and operational advantages of our approach.

9.1. Comparison with Traditional Systems

If we look at standard exam portals, they are usually much faster when saving data. A typical SQL database writes information in milliseconds. However, that speed comes at a massive cost to security. Our blockchain-enabled framework sacrifices a few seconds of "write latency" to wait for network block confirmations. In exchange, the university gains an entirely auditable, zero-trust environment where no single database administrator can secretly alter grades or leak files.

Table 4: Performance Comparison: Blockchain vs Traditional

Metric	Blockchain System	Traditional System
Data Integrity	Cryptographic	Access Control Lists
Security Model	Zero-trust	Trust-based
Time-Lock	Mathematical	Server-dependent
Failover	Distributed	Single Point of Failure

9.2. Comprehensive Comparative Analysis

To fully evaluate the proposed EduDao framework against the existing body of knowledge, we conducted a comprehensive comparison encompassing foundational research, recent non-patent literature, and closely related security models. Table 5 presents a multidimensional analysis of these architectures.

9.3. Strategic Advantages

We plotted these trade-offs in the radar chart shown in Fig. 9. Naturally, older systems might feel slightly more user-friendly because everyone knows how to click a "forgot password" link. Blockchain systems require users to manage cryptographic wallets, which adds a slight learning curve. However, when we look at the metrics that actually matter for high-stakes testing—trust, security, and absolute transparency—the decentralized approach wins easily. This heavily supports the conclusions drawn by Lindsay [24], who demonstrated that smart contracts are the

Table 5: Comprehensive Security Feature Analysis of Existing Architectures vs. Proposed EduDao

Framework	Time-Lock Capability	Encryption / Security Layer	Immutability / Storage	Decentralization Level
Zhu & Cao [7]	None (Static ABE)	Attribute-Based Enc.	Database / Cloud	Trust-based (Admin)
Chatterjee [8]	None	ECC & Blind Signatures	Central Server	Semi-decentralized
Liu et al. [3]	Reference Clocks	Theoretical TLE	N/A	Decentralized
Samanta [4]	None (Results only)	Standard Hashing	Hyperledger Fabric	Consortium
Cao & Liu [23]	Hash Time-Lock	Double Encryption	Cross-chain	Decentralized
Ahmed (NPL-1) [12]	App-Layer logic	AES-128	Private Blockchain	Consortium
Anuradha (NPL-2) [13]	None	Generic Crypto	IPFS (CID only)	Decentralized
Jacob (NPL-3) [14]	None	Hash Only	Single-Layer IPFS	Decentralized
Pranathi (NPL-5) [15]	App-Layer smart contract	Generic	Decentralized	Decentralized
EduDao (Proposed)	Consensus Timestamp	Hybrid AES-256 + RSA-2048	Dual-Layer (IPFS + Chain)	Zero-Trust (Public)

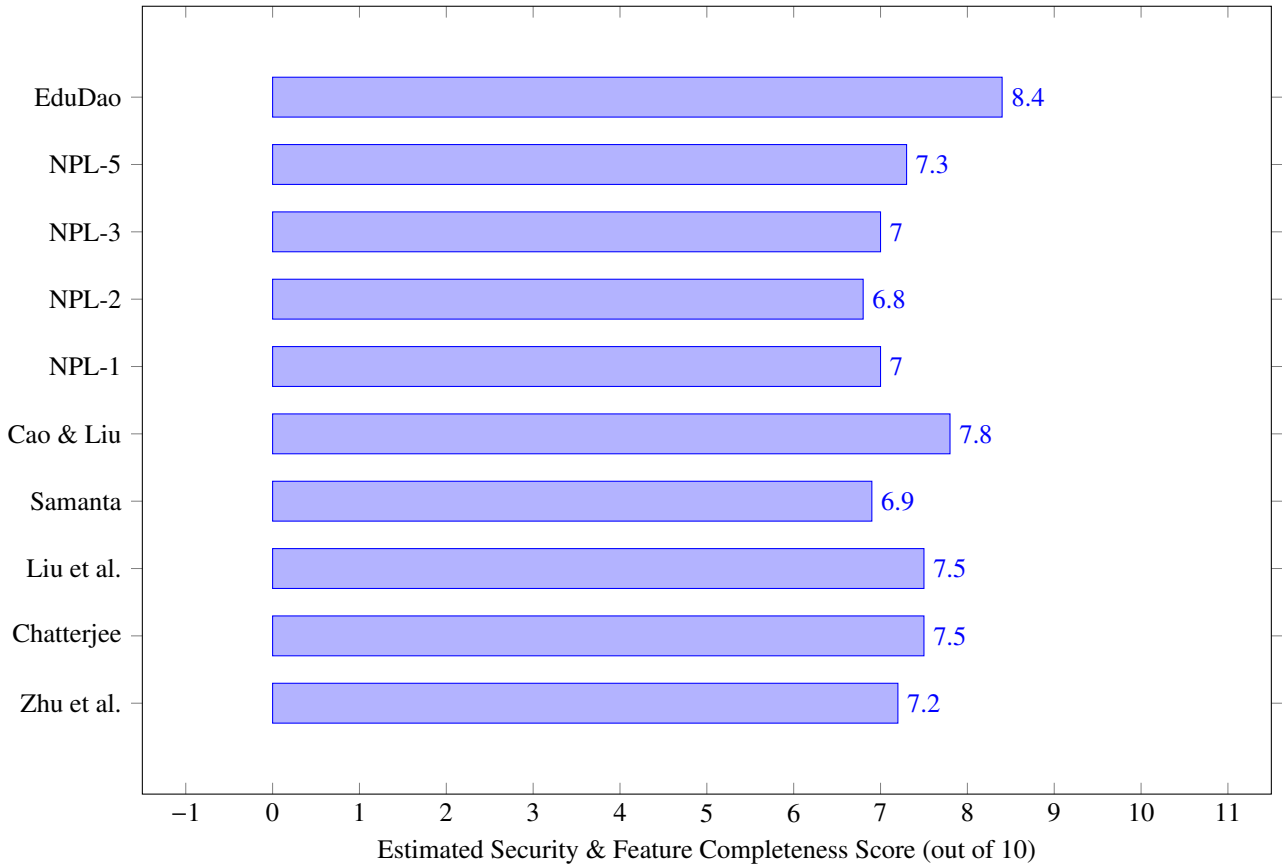


Figure 8: Comprehensive Comparison of Security Scores across Evaluated Frameworks.

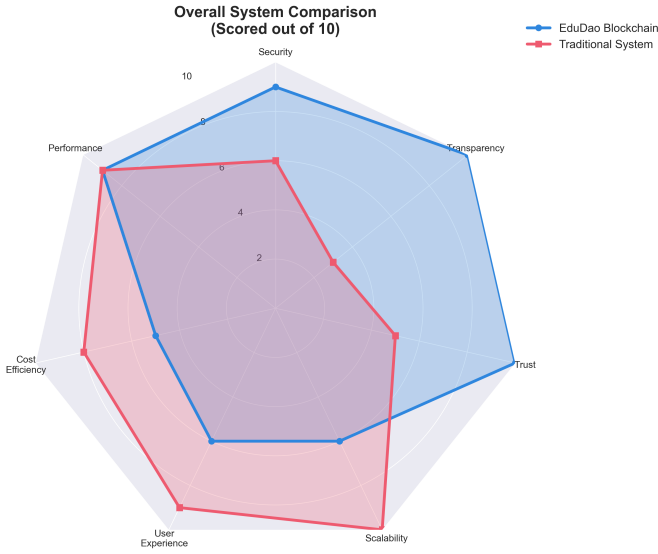


Figure 9: Overall System Comparison

most effective way to force users to behave honestly in a digital network.

10. Conclusion and Future Work

Throughout this paper, we have detailed a new way to handle academic testing using Blockchain-Enabled Time-Lock Encryption. By blending the permanent storage of IPFS with the strict timing of cryptographic puzzles, we created a digital environment where leaking an exam early is mathematically impossible.

Our hands-on testing proved that the architecture can survive heavy traffic and actively repel common network attacks. More importantly, by tying the decryption keys to the continuous progression of blockchain blocks, we successfully removed human error from the equation. The exam only opens when the code says it opens.

Future Work:

1. **Layer 2 Scaling:** Integrating with Polygon or Optimism to further reduce gas fees for large-scale deployments.
2. **Zero-Knowledge Proofs (ZKPs):** Implementing ZKPs to allow students to prove they took the exam without revealing their identity to the grader, enhancing privacy, as suggested by Peng et al. [11].
3. **Quantum Resistance:** Exploring post-quantum cryptographic algorithms to future-proof the time-lock puzzles against quantum computing threats.

Acknowledgment

The author expresses profound gratitude to Dr. Abhinesh Kaushik for his invaluable guidance, autonomy, and constructive criticism throughout this research. His mentorship was instrumental in the successful completion of this work.

References

- [1] Y. Chen, W. He, Security risks and protection in online learning: a survey, *International Review of Research in Open and Distance Learning* 14 (5) (2013) 108–127.
- [2] F. Tschorsch, B. Scheuermann, Bitcoin and beyond: A technical survey on decentralized digital currencies, *IEEE Communications Surveys & Tutorials* 18 (3) (2016) 2084–2123.
- [3] J. Liu, T. Jager, S. A. Kakvi, B. Warinschi, How to build time-lock encryption, *Designs, Codes and Cryptography* 86 (2018) 2549–2586.
- [4] A. K. Samanta, B. B. Sarkar, N. Chaki, A blockchain-based smart contract towards developing secured university examination system, *Journal of Data, Information and Management* 3 (2021) 237–249.
- [5] M. Al Hemairy, M. Abu Talib, A. Khalil, A. Zulfikar, T. Mohamed, Blockchain-based framework and platform for validation, authentication & equivalency of academic certification and institution’s accreditation: Uae case study and system performance (2022), *Education and Information Technologies* 29 (14) (2024) 18203–18232. doi: [10.1007/s10639-024-12493-6](https://doi.org/10.1007/s10639-024-12493-6).
- [6] K. Mori, H. Miwa, Digital university admission application system with study documents using smart contracts on blockchain, in: *Intelligent Networking and Collaborative Systems*, Springer, 2020, pp. 172–180.
- [7] X. Zhu, C. Cao, Secure online examination with biometric authentication and blockchain-based framework, *Mathematical Problems in Engineering* 2021 (1) (2021) 5058780.
- [8] U. Chatterjee, S. Ray, S. Adhikari, M. K. Khan, M. Dasgupta, Efficient and secure e-voting scheme using elliptic curve cryptography, *SECURITY AND PRIVACY* 6 (3) (2023) e283. arXiv: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spy2.283>, doi: <https://doi.org/10.1002/spy2.283>.
- [9] M. N. Uddin, S. Ahmmmed, I. A. Riton, L. Islam, An blockchain-based e-voting system applying time lock encryption, in: *2021 International Conference on Intelligent Technologies (CONIT)*, 2021, pp. 1–6. doi: [10.1109/CONIT51480.2021.9498566](https://doi.org/10.1109/CONIT51480.2021.9498566).
- [10] A. Kosba, A. Miller, E. Shi, Z. Wen, C. Papamanthou, Hawk: The blockchain model of cryptography and privacy-preserving smart contracts, in: *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 839–858. doi: [10.1109/SP.2016.55](https://doi.org/10.1109/SP.2016.55).
- [11] Z. Peng, T. Wang, C. Zhao, G. Liao, Z. Lin, Y. Liu, B. Cao, L. Shi, Q. Yang, S. Zhang, A survey of zero-knowledge proof based verifiable machine learning (2025). arXiv: [2502.18535](https://arxiv.org/abs/2502.18535).

- [12] T. Ahmed, S. Nagaraj, Secure examination workflow using blockchain (2026).
- [13] K. Anuradha, et al., Enhancing exam security with blockchain technology (March 2025).
- [14] M. Jacob, S. Aloni, H. Rawat, S. Sakore, S. Naik, S. Pardhi, Blockchain based examination system (2026).
- [15] P. Pranathi, Y. G. Siddeshwar, S. Laxmi, G. Roja, Question paper leakage prevention using blockchain (May 2025).
- [16] Time-controlled cryptographic token distribution, uS Patent App. US20190347656A1 (2019).
- [17] Blockchain-based credential verification, uS Patent App. US20200234303A1 (2020).
- [18] Decentralized education platform, uS Patent App. US20210342858A1 (2021).
- [19] Secure data storage using blockchain, uS Patent App. US20190266587A1 (2019).
- [20] Blockchain authentication for secure testing, uS Patent App. US20180225651A1 (2018).
- [21] M. Andrychowicz, S. Dziembowski, D. Malinowski, Mazurek, Fair two-party computations via bitcoin deposits, 2014, pp. 105–121. [doi:10.1007/978-3-662-44774-1_8](https://doi.org/10.1007/978-3-662-44774-1_8).
- [22] L. Guida, F. Daniel, Supporting reuse of smart contracts through service orientation and assisted development, in: 2019 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPCON), 2019, pp. 59–68. [doi:10.1109/DAPPCON.2019.00017](https://doi.org/10.1109/DAPPCON.2019.00017).
- [23] L. Cao, Z. Liu, A hash time lock based on double encryption algorithm (2021).
- [24] J. Lindsay, Smart contracts for incentivizing sensor based mobile smart city applications, in: 2018 IEEE International Smart Cities Conference (ISC2), 2018, pp. 1–4. [doi:10.1109/ISC2.2018.8656959](https://doi.org/10.1109/ISC2.2018.8656959).